ES Modules

10 min read · Updated 24 Jul 2026 · Topic 12 of 70

Why it matters

ES Modules (ESM) are the official, standardized module system of JavaScript — the same `import` and `export` syntax that runs in browsers. For years Node had only CommonJS; today Node fully supports ESM, and a large and growing slice of the ecosystem is ESM-only or dual-published. If you are writing new backend code, publishing a library, or sharing code between a Node server and a browser bundle, you need to be fluent in ESM. Just as importantly, the boundary between ESM and CommonJS is where a huge number of real-world errors live — `ERR_REQUIRE_ESM`, `Cannot use import statement outside a module`, `__dirname is not defined`. Understanding both systems and how they interoperate is what makes those errors instantly diagnosable instead of terrifying.

ESM is not just "new syntax for the same thing." It changes *when* dependencies are resolved (statically, before execution), *how* bindings behave (live, not copied), and *what* is available (top-level `await`, but no `__dirname`). Those differences have practical consequences the moment you switch.

The concept, simply

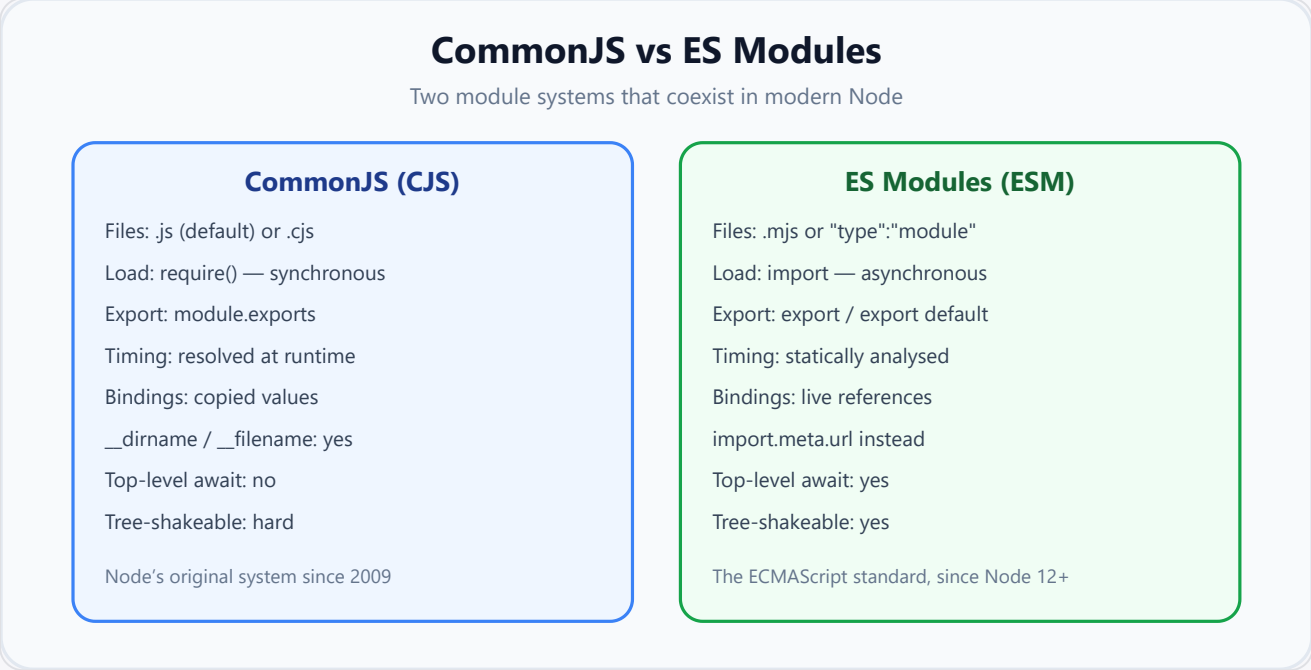
Instead of `require` / `module.exports`, ESM uses declarative `import` and `export` statements:

JAVASCRIPT Copy

```
// math.mjs
export function add(a, b) { return a + b; }
export const PI = 3.14159;
export default function multiply(a, b) { return a * b; }
```

```
import multiply, { add, PI } from './math.mjs';
```

The key mental shift: `import` / `export` are not function calls that run line by line — they are *static declarations* that the engine reads before any code executes. Node parses the whole dependency graph first, links it, and only then runs module bodies. That is why `import` must appear at the top level (not inside an `if`) and why the specifier must be a string literal, not a computed expression. For genuinely dynamic loading, ESM offers `await import(expr)`, which is an async function returning a promise.



CommonJS and ES Modules differ in more than syntax: loading model (sync vs async), binding semantics (copied values vs live references), and available globals all change when you switch.

How it works

Telling Node a file is ESM. Node decides a file's module system by three signals, in order: a `.mjs` extension is always ESM; a `.cjs` extension is always CommonJS; a plain `.js` file follows the nearest `package.json`'s `"type"` field — `"type": "module"` makes `.js` ESM, `"type": "commonjs"` (or absent) makes it CommonJS. So the same `.js` source can be interpreted either way depending on the package it lives in.

Three phases: parse, link, evaluate. ESM loading is not a single synchronous read like `require`. Node (1) *parses* every module in the graph to discover its imports and exports without running any code, (2) *links* them — connecting each import to the exact

makes circular ESM graphs behave more predictably than CommonJS ones.

Live bindings, not copies. A CommonJS `require` gives you a snapshot of `module.exports` at require time. An ESM `import` gives you a *live, read-only view* of the exporter's binding. If the exporting module later reassigns an exported `let`, importers see the new value. You cannot reassign an imported binding — imports are constant references into the source module.

JAVASCRIPT Copy

```
// counter.mjs
export let count = 0;
export function inc() { count++; }

// app.mjs
import { count, inc } from './counter.mjs';
console.log(count); // 0
inc();
console.log(count); // 1  - live binding reflects the change
// count = 5;       // TypeError: Assignment to constant variable-like bin
```

Top-level await. ESM module bodies may use `await` at the top level. Node treats such a module as asynchronous: any module importing it waits for its top-level awaits to settle before its own body runs. This is impossible in CommonJS and is a major reason `require` cannot synchronously load an ESM module that uses it.

Evaluated once, then cached. Like CommonJS, ESM keeps a module map (the module registry) keyed by the resolved URL, so a given module's body runs exactly once per process no matter how many files import it. The difference is what the cache holds: CommonJS caches the finished `module.exports` object, whereas ESM caches a *Module Record* whose exported bindings stay live. Import the same module from ten places and every importer shares one instance — the same singleton guarantee that makes a shared config or connection pool work, and the same trap if you mutate module-level state expecting isolation. One subtlety: the registry key is the fully-resolved URL including any query string, so `import './x.js?v=1'` and `import`

`import.meta` **— per-module metadata.** `import.meta` is an object the host populates with information about the current module. Node always provides `import.meta.url` (the module's absolute `file:` URL) and, on Node 20.11+/21.2+, the convenience properties `import.meta.dirname` and `import.meta.filename`. It also exposes `import.meta.resolve(specifier)`, the ESM counterpart to `require.resolve`, which returns the URL a specifier would resolve to without loading it — handy for locating an asset shipped beside your code. Because `import.meta.url` is a real URL string, the idiomatic way back to a filesystem path is always `fileURLToPath(import.meta.url)`; slicing off the `file://` prefix by hand breaks on Windows drive letters and on paths containing spaces or non-ASCII characters, which is exactly why the `node:url` helper exists.

No `__dirname`, no `require`. ESM files do not get the CommonJS wrapper, so `require`, `module`, `exports`, `__filename`, and `__dirname` do not exist. Instead you get `import.meta.url` (the module's URL). Reconstruct the old values with the `url` module, or on modern Node use the built-in helpers:

JAVASCRIPT Copy

```
import { fileURLToPath } from 'node:url';
import path from 'node:path';

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);
// Node 20.11+ / 21.2+ also expose import.meta.dirname and import.meta.fil
```

ESM ↔ CJS interop. The two systems meet under clear rules:

- **ESM importing CJS:** allowed. The CommonJS `module.exports` object becomes the ESM *default* export. So `import pkg from 'some-cjs-lib'` gives you `module.exports`. Node also tries to statically detect named exports (`import { readFile } from 'node:fs'` works for builtins and many packages), but detection is best-effort — when it fails, import the default and destructure.

works in both worlds. (Recent Node versions can `require()` a synchronous ESM module — one with no top-level await — but `import()` remains the portable answer.)

- File extensions are mandatory in relative ESM specifiers: `import './util.js'`, not `import './util'`. CommonJS's extension guessing does not apply.

Code examples

1. Named, default, and namespace imports.

JAVASCRIPT Copy

```
// shapes.mjs
export const PI = 3.14159;
export function area(r) { return PI * r * r; }
export default class Circle { constructor(r) { this.r = r; } }

// app.mjs
import Circle, { area, PI } from './shapes.mjs'; // default + named
import * as shapes from './shapes.mjs';          // namespace object
console.log(shapes.PI, shapes.default);          // 3.14159, [class Circle
```

2. Top-level await at module scope.

JAVASCRIPT Copy

```
// config.mjs
const res = await fetch('https://example.com/config.json');
export const config = await res.json();

// server.mjs — waits for config.mjs's top-level awaits before running
import { config } from './config.mjs';
console.log('Loaded config:', config);
```

3. Loading a CommonJS module from ESM.

```
// legacy-lib is CommonJS: module.exports = { greet }
import legacy from 'legacy-lib';   // whole module.exports as default
legacy.greet('world');
// Named import may also work if Node detects it:
import { greet } from 'legacy-lib'; // best-effort static analysis
```

4. Loading ESM from CommonJS with dynamic import.

JAVASCRIPT Copy

```
// index.cjs (CommonJS)
async function main() {
  const { default: chalk } = await import('chalk'); // chalk is ESM-only
  console.log(chalk.green('works from CJS via import()'));
}
main();
```

Common mistakes & gotchas

- **Omitting the file extension.** `import './util'` throws `ERR_MODULE_NOT_FOUND` in ESM — write `'./util.js'`.
- **Using `__dirname` in ESM.** Not defined — reconstruct from `import.meta.url` or use `import.meta.dirname`.
- **`require` of an ESM package.** Classic `ERR_REQUIRE_ESM`. Use `await import()` instead, or make your own file ESM.
- **Forgetting `"type": "module"`.** A `.js` file using `import` in a default package throws `Cannot use import statement outside a module`.
- **Expecting `import` to run conditionally.** Static `import` is hoisted and always evaluated. For conditional/dynamic loading use `await import()`.
- **Assuming named imports always exist on a CJS package.** Node's named-export detection is heuristic; import the default and destructure when it fails.
- **Trying to reassign an import.** Imports are read-only live bindings — assignment is a syntax/type error.

- Mixing `module.exports` and `export` in one file. A file is one system or the other; you cannot use both.

Best practices

- For new projects, prefer ESM: set `"type": "module"` and use `.js` with `import` / `export`.
- Always include explicit file extensions in relative imports.
- Replace `__dirname` with `import.meta.dirname` (or `fileURLToPath(import.meta.url)` for older Node).
- When publishing a library, ship **dual** builds and wire them with the `exports` field's `import` / `require` conditions so both worlds work.
- Use `await import()` as the portable bridge whenever you must load ESM from CJS or load a module conditionally/lazily.
- Keep top-level `await` deliberate — it delays every importer; do not use it for work that could be a lazily-called async function.
- Prefer named exports for tree-shakeable, discoverable APIs; reserve `default` for a single primary export.

⭐ Interview lens

Q: Name three concrete differences between ESM and CommonJS beyond syntax. (1) Loading: `require` is synchronous and runtime; `import` is statically analysed and async. (2) Bindings: CJS gives a copied snapshot of exports; ESM gives live read-only bindings. (3) Environment: ESM has top-level `await` and `import.meta` but no `__dirname` / `require`.

Q: What does "live binding" mean? An imported name is a reference into the exporting module's variable, not a copy. If the exporter reassigns an exported `let` later, importers observe the new value. In CommonJS you would have captured the old value at require time.

`package.json`'s `"type"` field: `"module"` → ESM, `"commonjs"` or absent → CJS. `.mjs` is always ESM and `.cjs` always CJS regardless of `"type"`.

Q: You get `ERR_REQUIRE_ESM`. What is happening and how do you fix it? Code is calling `require()` on an ESM-only package. Options: convert the calling file to ESM, or use `const mod = await import('pkg')` which works from both systems. Trap: interviewers may ask why you can't "just require it" — because ESM may use top-level await, which cannot resolve synchronously.

Q: How do you get `__dirname` in an ES module? `const __dirname = path.dirname(fileURLToPath(import.meta.url))`, or directly `import.meta.dirname` on Node 20.11+.

Q: When you `import x from` a CommonJS module, what is `x`? The module's entire `module.exports` object, bound as the default export. Named imports may additionally work if Node's static analysis detects them.

Cheat-card

- **ESM signals:** `.mjs`, or `.js` + `"type":"module"`. **CJS:** `.cjs`, or `.js` default.
- **Syntax:** `export` / `export default` / `import x, { y } from './f.js'`.
- **Extensions required** in relative ESM specifiers.
- **Live bindings** (read-only) vs CJS copied snapshot.
- **Top-level await** allowed in ESM only; makes the module async.
- **No** `__dirname` → use `import.meta.dirname` / `fileURLToPath(import.meta.url)`.
- **Interop:** ESM imports CJS as default; CJS loads ESM via `await import()`.
- `await import(expr)` = dynamic/conditional loading, works in both systems.

Previous
CommonJS Modules

Next
Module Resolution

LEARN MORE

ARTICLES & DOCS

- Modules: ECMAScript modules
  nodejs.org
- JavaScript modules (import / export)
  developer.mozilla.org

VIDEOS



ESM vs CommonJS in your Node Projects
Covalence

Hand-picked references for this lesson.